



INDIA.RUNS

Build what next India runs on

Team Name : Redrob AI Ranker Elite

Team Leader Name : Zia Bhombal

Problem Statement : AI-Powered Candidate Ranking Pipeline for 100,000 Profiles

Solution Overview

Our proposed solution is an end-to-end **Candidate Search & Ranking System** powered by a hybrid neural-heuristic ranking engine. It leverages **SentenceTransformer** semantic vector embeddings indexed using a high-speed **FAISS** retrieval layer, combined with a **52-feature weighted scoring engine** that evaluates career trajectory, technical capability, education relevance, platform engagement, and Redrob behavioral signals.

Unlike traditional recruitment systems that rely on rigid, error-prone keyword matching, our solution:

- **Employs dense vector embeddings** to capture the semantic similarity between job descriptions and candidate profiles, enabling context-aware matching beyond exact keywords.
- **Implements career stability, role progression, and educational tier scoring** to assess professional maturity and long-term growth potential.
- **Integrates 23 native Redrob behavioral signals** and platform activity metrics to prioritize highly engaged and responsive candidates.
- **Features an explainability engine** that generates human-readable reasoning for every shortlisted candidate, ensuring transparency and recruiter confidence in the ranking process.

JD Understanding & Candidate Evaluation

Our system evaluates candidate fit across the following signal dimensions:

- **Required and Preferred Skills:** Extracted using lexical hints and contextual rules.
- **Experience and Seniority:** Years of experience are parsed using regular expressions, while seniority is classified based on experience duration.
- **Professional Domain & Leadership:** Identifies domain expertise, leadership context, and role-specific requirements.
- **Semantic Alignment (FAISS Search Score):** Measures semantic similarity between candidate profile summaries and the job description (JD).
- **Career Progression:** Evaluates career stability (tenure/stability index) and the rank/tier of previous organizations.
- **Technical Depth:** Assesses skill rarity, total skill count, GitHub repository portfolio quality, and project recency.
- **Education Rank:** Considers degree level, institution ranking tier, and the relevance of the academic field.
- **Engagement Patterns:** Analyzes response velocity, profile consistency, network indicators, and behavioral signals.

Ranking Methodology

Hybrid Retrieval & Ranking Pipeline

We implement a hybrid retrieval-ranking pipeline using **sentence-transformers/all-MiniLM-L6-v2** (384-dimensional dense vectors). Multiple signals are combined into a final score (**0.0 to 1.0**) using a weighted linear ensemble.

Semantic Retrieval: Query the Job Description against a **100,000-candidate FAISS index** to retrieve the top **K** (e.g., 1,000) candidates.

Feature Engineering: Compute **52 normalized features** for the retrieved candidates.

Multi-Factor Scoring: Combine semantic match, career progression, technical capability, and behavioral signals into a weighted score.

Ranking: Sort candidates and return the final **Top 100** shortlist.

Score Composition:

Career Score (30%): Tenure, company tier, title progression.

Semantic Match (25%): FAISS cosine similarity.

Technical Capability (25%): GitHub metrics, skill volume, technical depth.

Engagement & Signals (20%): Redrob behavioral signals and response velocity.

Explainability & Data Validation

For every ranked candidate, our system generates a structured, natural-language explanation outlining their title, tenure, matching skills, and key scores (e.g., semantic match, experience fit, and behavioral fit) to provide transparent justification to recruiters.

Grounding: Explanations are deterministic and template-based, using only verified candidate attributes.

Validation: Detects profile anomalies (e.g., salary max < min, activity date before signup, missing dates).

Traceability: Every explanation maps directly to computed features from the candidate record.

Quality Flags: Appends warnings (e.g., temporal_inconsistency, salary_anomaly) and applies scoring penalties for low-quality data.

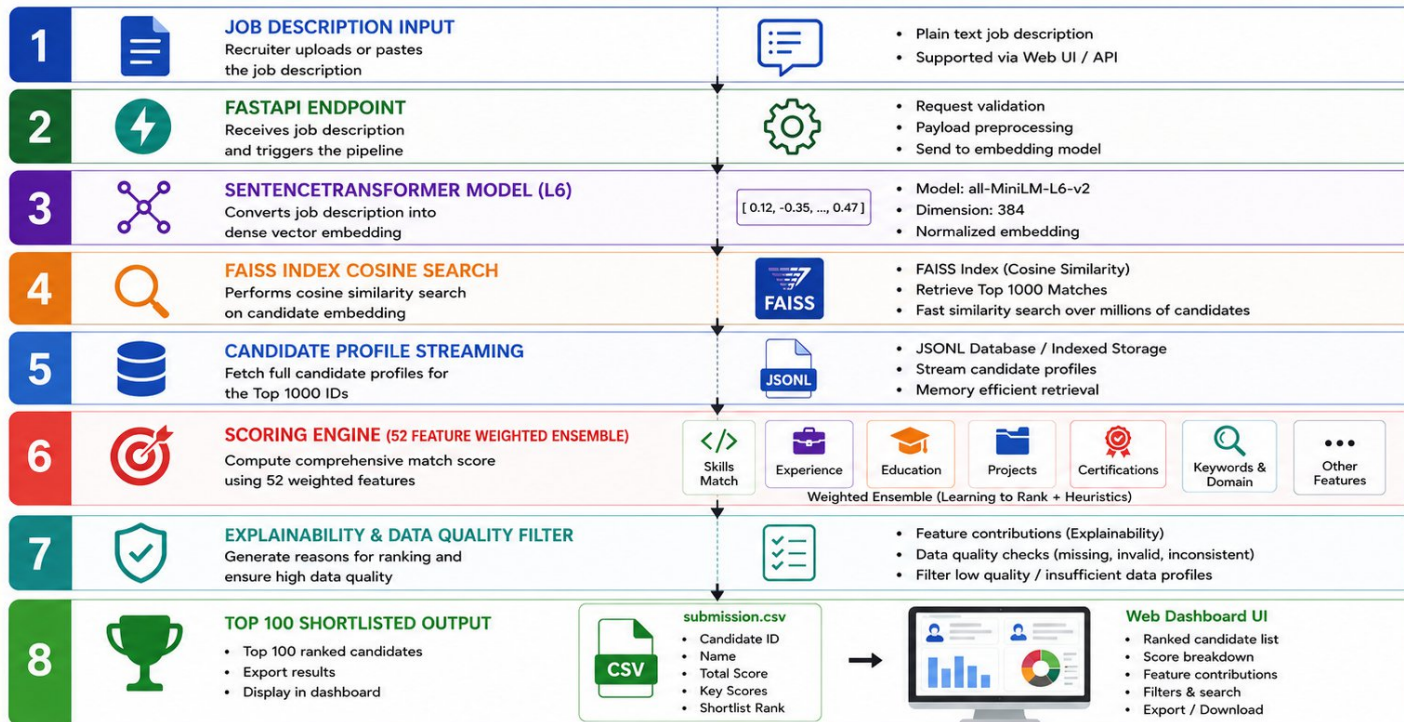
End-to-End Workflow

Our workflow operates in 8 sequential stages:

- Stage 1: Build technical skill data dictionary.
- Stage 2: Load and parse the Job Description.
- Stage 3: Validate and load the 100k candidate embeddings cache.
- Stage 4: Encode the Job Description using the SentenceTransformer model.
- Stage 5: Load the pre-built candidate FAISS index file.
- Stage 6: Perform semantic search against the FAISS index to retrieve the top 1000 candidates.
- Stage 7: Stream candidates from the database and score/rank them dynamically.
- Stage 8: Write the top 100 shortlisted candidates and reasoning to submission.csv.

System Architecture

AI RESUME SCREENING PIPELINE – COMPONENT ARCHITECTURE



TECHNOLOGY STACK

- FastAPI
- SentenceTransformers
- FAISS
- Python
- Pandas
- Scikit-learn
- Plotly / Chart.js
- HTML / CSS / JS

DATA STORAGE

- Candidate Profiles (JSONL)
- Embeddings Index (FAISS)

Results & Performance

Shortlist Quality: Top recommendations include stable, senior, and technically strong candidates from tier-1 companies (e.g., Yellow.ai, Meesho, Sarvam AI, Razorpay).

Rapid Search: Search, evaluation, and ranking across **100,000 candidates** complete in **under 3 seconds** using a warm FAISS index.

Signal Alignment: Recommended candidates closely match the required skills and seniority.

Memory Efficiency: Streams candidate data from **JSONL** instead of loading all records into RAM, keeping memory usage below **500 MB**.

Technologies Used

Python 3.13: Selected as the core programming language for processing and machine learning.

FastAPI & Uvicorn: A high-performance web framework to serve real-time candidate search and ranking API endpoints.

FAISS: The Facebook AI Similarity Search library, used for lightning-fast vector search over the 100,000 candidate profiles.

Sentence-Transformers (all-MiniLM-L6-v2): A highly optimized, lightweight text embedding model for semantic similarity representation.

Pytest: Professional testing framework utilized for pipeline and data quality validation.

HTML5 / CSS3 / Vanilla JS: A clean and responsive front-end dashboard with zero external library overhead to keep page load and rendering instant.

Submission Assets

- GitHub Repository: Clean, fully refactored, production-ready code with complete unit tests.
- Output Data: submission.csv containing top 100 ranked candidates with deterministic scores and reasons.
- Web Video Demo: Recorded walkthrough of the interactive dashboard (dashboard_success_flow.webp).
- Local Web URL: <http://127.0.0.1:8000/> serving the live application.



INDIA.RUNS

Build what next India runs on

THANK YOU